

Python Naming Conventions: Variables

In Python, like any programming language, **naming conventions** are crucial for writing clear, readable, and maintainable code. While some rules are strictly enforced by the language itself, others are widely accepted best practices that promote consistency and collaboration among developers. This guide focuses on naming conventions for variables in Python.

Strict Rules: What Python Accepts and Rejects

These are the fundamental rules that Python enforces for variable names. If you violate these, your code will result in a `SyntaxError`.

- 1. Must Start with a Letter or Underscore:** Variable names must begin with an uppercase or lowercase letter (a-z, A-Z) or an underscore (`_`).
 - **Accepted:** `my_variable` , `_internal_var` , `name`
 - **Rejected:** `1st_variable` (starts with a number)
- 2. Cannot Start with a Number:** As seen above, starting a variable name with a digit is not allowed.
- 3. Can Only Contain Alphanumeric Characters and Underscores:** After the first character, variable names can contain letters (a-z, A-Z), numbers (0-9), and underscores (`_`). No spaces, hyphens (`-`), or other special characters are permitted.
 - **Accepted:** `user_name_2` , `total_count`
 - **Rejected:** `user-name` , `total count` , `item#price`
- 4. Case-Sensitive:** Python is a case-sensitive language. This means `myVariable` and `myvariable` are treated as two distinct variables.
 - **Example:**

Python

```
age = 30
Age = 40
print(age) # Output: 30
print(Age) # Output: 40
```

- 5. Cannot Be Python Keywords:** You cannot use Python's reserved keywords (like `if` , `else` , `for` , `while` , `class` , `def` , `import` , `return` , etc.) as variable names. Doing so will also result in a `SyntaxError`.
 - **Accepted:** `my_if_statement`

- **Rejected:** `if = 10`

Best Practices: What is Recommended (PEP 8)

Beyond the strict rules, the Python community adheres to a style guide called **PEP 8** (Python Enhancement Proposal 8). Following PEP 8 makes your code consistent with the vast majority of Python projects, making it easier for others (and your future self) to read and understand.

1. **Use snake_case for Variables and Functions:** For variable names, PEP 8 recommends using lowercase letters with words separated by underscores. This is known as `snake_case`.
 - **Recommended:** `first_name` , `total_price` , `is_active`
 - **Avoid:** `firstName` (camelCase), `TOTALPRICE` (all caps, usually for constants)
2. **Choose Descriptive Names:** Variable names should be clear and descriptive, indicating their purpose or the data they hold. Avoid overly short or ambiguous names unless their context is immediately obvious (e.g., loop counters like `i` or `j`).
 - **Recommended:** `customer_age` , `product_quantity`
 - **Avoid:** `ca` , `pq` , `x` (unless `x` is a temporary variable in a very small scope)
3. **Avoid Single-Letter Names (Generally):** While allowed, single-letter names (except for specific cases like loop iterators or coordinates) can make code harder to understand.
4. **Avoid Leading/Trailing Underscores for Regular Variables:**
 - A single leading underscore (`_variable`): Conventionally indicates a variable intended for internal use within a module or class. It's a hint to other programmers not to access it directly, though Python doesn't strictly prevent it.
 - A double leading underscore (`__variable`): Used for name mangling in classes to avoid naming conflicts in subclasses.
 - A single trailing underscore (`variable_`): Used to avoid conflicts with Python keywords (e.g., `class_` instead of `class`).
5. **Avoid Ambiguous Characters:** Do not use the lowercase letter `l` , the uppercase letter `O` , or the uppercase letter `I` as single-character variable names. In some fonts, these can be easily confused with the number `1` or `0` .

Summary Table

Rule Category	Description	Accepted Examples	Rejected Examples	Best Practice Examples (PEP 8)

Starting Character	Must be a letter or <code>_</code>	<code>name</code> , <code>_count</code>	<code>1name</code> , <code>!count</code>	<code>user_name</code>
Allowed Characters	Alphanumeric and <code>_</code>	<code>user_id</code> , <code>item_price_2</code>	<code>user-id</code> , <code>item price</code>	<code>product_id</code>
Case Sensitivity	Python is case-sensitive	<code>age</code> and <code>Age</code> are different	N/A	<code>total_amount</code> (<code>snake_case</code>)
Keywords	Cannot be Python keywords	<code>my_if</code> , <code>class_name</code>	<code>if</code> , <code>class</code>	<code>is_valid</code>
Readability	Clear and descriptive	<code>temperature</code> , <code>is_logged_in</code>	<code>temp</code> , <code>flag</code>	<code>sensor_temperature</code>

By adhering to these rules and best practices, you'll write Python code that is not only functional but also elegant, readable, and easily understood by anyone who encounters it.

References

[1] Van Rossum, G., Warsaw, B., & Coghlan, N. (2001). PEP 8 -- Style Guide for Python Code. Python Software Foundation. Retrieved from